

SCM Training

Schema Change Manager — version control for your database

Audience: anyone who changes a database schema (developers, DBAs, analysts) **Goal:** by the end, every attendee can create, apply, verify, and roll back a schema change **Reference:**

[scm.md](#) (full manual) · [scm-readme.md](#) (standalone install) · `scm learn` (built-in cheat sheet)

Master Checklist

Work top to bottom. Each item has its own slide.

Concepts

- ☐ 1. Why SCM exists
- ☐ 2. Why not an existing tool
- ☐ 3. The mental model
- ☐ 4. What a migration is
- ☐ 5. How state is tracked

Getting set up (no WaSQL install required)

- ☐ 6. Get scm from the WaSQL repo
- ☐ 7. Put scm on your PATH
- ☐ 8. Install a database driver
- ☐ 9. Configure a database
- ☐ 10. Know which database you are pointed at

The everyday loop

- ☐ 11. Create a migration
- ☐ 12. Anatomy of a migration file
- ☐ 13. Check status before you act
- ☐ 14. Apply with up
- ☐ 15. Roll back with down
- ☐ 16. Jump to a version with goto

Visibility and auditing

- [] 17. Inspect with show, history, report, ddl

Special situations

- [] 18. Adopt an existing database with baseline
- [] 19. The careful commands: undo, repair, reset
- [] 20. Many databases, one migration set

Discipline

- [] 21. Writing safe migrations
- [] 22. Database specific gotchas
- [] 23. Credentials, permissions, and CI
- [] 24. AI ready: using SCM with Claude

Practice

- [] 25. Hands on lab
- [] 26. The five rules to remember

1. Why SCM exists

Schema changes run by hand do not scale. They get forgotten, run twice, run in the wrong order, or run in dev and never in prod. There is no history of who changed what, and "undo" is a guess.

SCM makes every schema change a **plain SQL file in git**, applied in a known order and **tracked in the database itself**. Any environment can be brought up to date by running one command, and only the missing changes are applied.

Without SCM	With SCM
SQL pasted into a client, then lost	SQL lives in a file, in git, reviewable
"Did prod get that ALTER?"	<code>scm status</code> answers it
Environments silently drift	Same files, same result everywhere
Rollback is improvised	Every change ships with a <code>down</code>
No idea who ran what	<code>scm report</code> names names and times
AI fires DDL straight at the DB	AI writes a migration you review first

Talking point: SCM is not a new SQL dialect and not an ORM. It is plain SQL plus bookkeeping.

2. Why not an existing tool

Migration tools are not new. The problem is that **we do not run one database**. We run MySQL, PostgreSQL, SQL Server, Oracle, **SAP HANA**, Snowflake, and **FairCom cTree** — and no existing tool covers that set. Adopting the popular tools means learning three of them, each with its own file format, and still hand running SQL on the engines none of them support.

Database coverage

Database	scm	dbmate	golang-migrate	Flyway	Liquibase	Alembic	Sqitch
PostgreSQL	✓	✓	✓	✓	✓	✓	✓
MySQL / MariaDB	✓	✓	✓	✓	✓	✓	✓
SQLite	✓	✓	✓	✓	✓	✓	✓
SQL Server	✓		✓	<i>paid</i>	✓	✓	
Oracle	✓			<i>paid</i>	✓	✓	✓
SAP HANA	✓			<i>paid</i>	<i>paid</i>	<i>dialect</i>	
Snowflake	✓		<i>community</i>	<i>paid</i>	<i>paid</i>	<i>dialect</i>	✓
FairCom cTree	✓						
Firebird	✓		<i>community</i>	<i>paid</i>	✓	<i>dialect</i>	✓
All of the above, one tool	✓						

✓ = supported out of the box · blank = not supported · *paid* = vendor's paid edition · *community* = community maintained driver · *dialect* = only via a third party SQLAlchemy dialect. Coverage moves over time — check current docs before quoting a competitor. The row that matters is the last one.

How they differ beyond coverage

	scm	dbmate	golang-migrate	Flyway	Liquibase	Alembic
Migration format	plain SQL	plain SQL	plain SQL	plain SQL + Java	XML / YAML / JSON / SQL	Python scripts
Runtime needed	Python 3.8+	Go binary	Go binary	JVM	JVM	Python + SQLAlchemy
Licensing	free	free	free	tiered	tiered	free
Both file styles	yes	one file only	two file only	its own	its own	its own
Runs from any folder	yes, one install	yes	yes	per project config	per project config	per project config
Add a new database yourself	~40 lines of Python	no	fork it	no	plugin (Java)	via SQLAlchemy dialect
Interoperable tracking table	yes, dbmate compatible	—	own	own	own	own
AI ready	rules shipped in repo	no	no	no	no	no

The actual argument

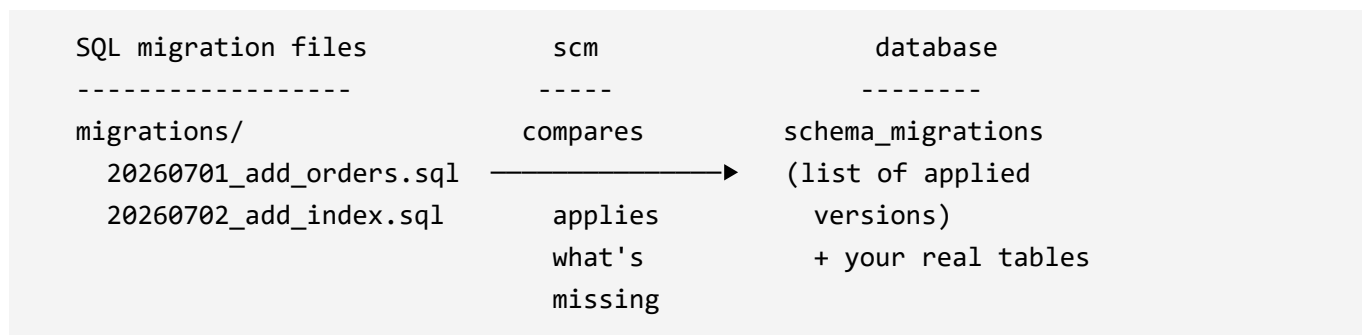
1. **One tool, every database we own** — including HANA and cTree, which nothing else supports. Same commands whether you are migrating MySQL or HANA; only the connection string changes.
2. **Plain SQL, no DSL.** Nobody writes schema changes in XML. What you review is what runs.
3. **No runtime to install and no license tier.** One Python file, ~2,300 lines, in the repo. No JVM, no vendor portal, no per engine upgrade.
4. **Nothing to migrate to it.** The `schema_migrations` table is dbmate compatible, so a project already tracked by dbmate keeps working, and `baseline` adopts a database that

was never tracked at all.

5. **Extensible in an afternoon.** A new engine is a `BaseDriver` subclass with five methods. That is why HANA and cTree exist here and nowhere else.
6. **AI ready.** Claude already knows how to drive it — the rules live in the repo ([scm.md](#) → *Rules for Claude*), so "add an orders table" produces a branch, a proper migration file, and a `down`, instead of a `CREATE TABLE` fired straight at production. See slide 24.

Talking point: we did not build this because dbmate is bad. We built it because half our databases were not in *anyone's* list.

3. The mental model



Three moving parts, nothing more:

1. **A folder of migration files** — versioned by timestamp, committed to git.
2. **The `scm` command** — compares files on disk to what the database says it has.
3. **A tracking table in the database** — `schema_migrations`, the source of truth for "what has already run here."

Because the comparison happens per database, the *same* files produce the *same* schema in dev, stage, and prod.

4. What a migration is

A migration is **one change**, expressed twice:

- **up** — how to apply it
- **down** — how to reverse it

```
-- migrate:up
CREATE TABLE IF NOT EXISTS orders (
```


Term	Meaning	What to do
pending	File on disk, no row in the table	Run <code>scm up</code>
orphaned	Row in the table, file is gone	Investigate, then <code>scm repair</code>

Two details that save future confusion:

- **Self healing.** On an older or hand built table, scm adds the missing `name / applied_by / migrated_by` columns (and `applied_at` where the database allows it) on the next run. `scm ddl` verifies the table.
- **PostgreSQL schema pinning.** An unqualified table lands wherever `search_path` points, which differs per user and environment, so history appears to vanish. scm therefore pins the table to `public` by default. Override with `MIGRATIONS_SCHEMA` in `.env`.

Demo: run `scm up`, then `SELECT * FROM schema_migrations` and show the new row. Then `scm up` again and show that nothing happens.

6. Get scm from the WaSQL repo

You do not need WaSQL installed. SCM lives in the WaSQL repo but is completely standalone — one Python file. Grab just that file.

Everything required:

File	Needed?	Purpose
<code>scm.py</code>	Yes	The entire tool
<code>scm.bat</code>	Windows	Lets you type <code>scm</code> instead of <code>python scm.py</code>
<code>scm.sh</code>	Optional (Unix)	Wrapper; or call <code>scm.py</code> directly via its shebang
<code>scm.md</code>	Optional	The manual

Option A — Download the two files (simplest, no git)

Windows (PowerShell)

```
New-Item -ItemType Directory -Force "$HOME\scm" | Out-Null
Set-Location "$HOME\scm"
```

```
$base = 'https://raw.githubusercontent.com/WaSQL/php/master/python'
Invoke-WebRequest "$base/scm.py" -OutFile scm.py
Invoke-WebRequest "$base/scm.bat" -OutFile scm.bat
```

Linux / macOS

```
mkdir -p ~/.local/scm && cd ~/.local/scm
curl -O https://raw.githubusercontent.com/WaSQL/php/master/python/scm.py
chmod +x scm.py
```

Option B — Sparse checkout (recommended, updates via `git pull`)

Clones **only** the `python/` folder, not the whole WaSQL repo:

```
git clone --filter=blob:none --sparse https://github.com/WaSQL/php.git wasql-scm
cd wasql-scm
git sparse-checkout set python
# scm now lives in ./python – update anytime with: git pull
```

Option C — Already have a WaSQL checkout

Copy `scm.py` (and `scm.bat` on Windows) out of its `python/` folder to wherever you want.

Updating is just "replace the file." `scm.py` is self-contained: Option A means re-running the download, Option B means `git pull`. Your `.env` files and `migrations/` folders are never touched by an update.

Keep `scm.py` and its wrapper **in the same folder** — the wrapper looks for `scm.py` right beside itself.

7. Put scm on your PATH

This is what lets you type `scm` from inside **any** project.

scm is context aware: the `.env` file and the `migrations/` folder are resolved from your **current working directory**, never from where `scm.py` lives. So one install serves every project on your machine.

Windows — `.bat` is on `PATHEXT`, so adding the folder is enough for both CMD and PowerShell:


```
[Environment]::SetEnvironmentVariable(
    'Path',
    [Environment]::GetEnvironmentVariable('Path', 'User') + ";$HOME\scm",
    'User')
```

Then **restart your terminal**. (GUI alternative: `sysdm.cpl` → Advanced → Environment Variables → edit **Path** under *User variables*.)

Linux / macOS — `scm.py` has a `#!/usr/bin/env python3` shebang, so symlink the **.py file** itself:

```
mkdir -p ~/.local/bin
chmod +x ~/.local/scm/scm.py
ln -sf ~/.local/scm/scm.py ~/.local/bin/scm
# ensure ~/.local/bin is on PATH (add to ~/.bashrc or ~/.zshrc):
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc && source ~/.bashrc
```

Verify in a new terminal — this is the checkpoint for the room:

```
scm version      # scm.py 1.29.0
scm learn        # the full quick-start reference
```

Do **not** symlink `scm.sh` / `scm.bat` onto your PATH while leaving `scm.py` elsewhere — the wrapper then looks for the script in the wrong place. Symlink `scm.py` directly, or put the whole folder on PATH.

Full install walkthrough, per OS, plus install troubleshooting: [scm-readme.md](#).

8. Install a database driver

Python 3.8+ plus one driver package is the entire dependency list. SCM itself uses only the standard library. Install only what you actually connect to:

Database	Install
SQLite	<i>nothing — built into Python</i>
PostgreSQL	<code>pip install psycopg2-binary</code>
MySQL / MariaDB	<code>pip install mysql-connector-python</code> (or <code>pymysql</code>)

Database	Install
SQL Server	<code>pip install pyodbc</code>
Oracle	<code>pip install oracledb</code>
SAP HANA	<code>pip install hdbcli</code>
Snowflake	<code>pip install snowflake-connector-python</code>
FairCom cTree	<code>pip install pyodbc</code> + FairCom ODBC driver
Firebird	<code>pip install fdb</code>

MySQL tries `mysql-connector-python` first and falls back to `pymysql`, so either works.

For today's lab you need nothing at all — the exercise uses SQLite.

If you hit `No <database> driver found`, the package went into a different Python than the one running scm. Install with `python3 -m pip install ...` to be sure.

9. Configure a database

One time per database, run from inside the project you want to manage:

```
scm init          # creates ./migrations and a .env stub
```

Then set the connection in `.env`:

```
DATABASE_URL=mysql://user:pass@localhost/mydb
MIGRATION_STYLE=one
# MIGRATIONS_DIR=./migrations
# MIGRATIONS_TABLE=schema_migrations
# MIGRATIONS_SCHEMA=public
```

One line per database you work with:

```
DATABASE_URL=postgres://user:pass@host:5432/mydb
DATABASE_URL=mysql://user:pass@localhost:3306/mydb
DATABASE_URL=mssql://user:pass@localhost:1433/mydb
DATABASE_URL=oracle://user:pass@host:1521/mysevice
DATABASE_URL=hana://user:pass@host:39015/
DATABASE_URL=sqlite://demo.db
```

The full URL format for every supported engine is in [scm.md](#) . Precedence is `--url` **flag** → **real environment variable** → `.env` .

Never commit real credentials. `.env` belongs in `.gitignore` . Prefer `.env` over the `--url` flag, which leaks the password into shell history and process lists.

10. Know which database you are pointed at

The single highest value habit in this training. Run it before anything destructive.

```
scm who          # what am I connected to?
scm dbs          # what databases are configured here?
```

Current database

```
--db      demo
env file   .env.demo
driver     sqlite
host       demo.db
tracking   schema_migrations
url        sqlite://demo.db
```

--db	env file	driver	host	database
(default)	.env	mysql	localhost	mydb
hana-t2	.env.hana-t2	hana	hana-t2:39015	
ods	.env.ods	postgres	ods-host:5432	ods

3 env file(s) found. Use `--db <name>` to target one.

Neither command connects to the database, and passwords are always masked. `scm dbs` is your menu of `--db` names; `scm who` confirms which one is currently in effect.

11. Create a migration

Never hand name a migration file. Let scm stamp the timestamp.

```
scm new add_orders_table
# Created migrations/20260701093000_add_orders_table.sql
```

- Names may contain letters, digits, `_` and `-`, and must start with a letter or digit.
- Timestamps never collide.** Run `new` five times in one second and you get `...12`, `...13`, `...14`, `...15`, `...16` automatically.
- `--style two` produces the two file variant (`.up.sql` + `.down.sql`).

Recommended team habit — branch first, so the migration is reviewed like any other code:

```
git checkout -b migration/add_orders_table
scm new add_orders_table
```

12. Anatomy of a migration file

Two styles are supported; both can coexist in one folder.

Single file (default, dbmate compatible) — `20260701093000_add_orders.sql`

```
-- migrate:up
CREATE TABLE IF NOT EXISTS orders (...);

-- migrate:down
DROP TABLE IF EXISTS orders;
```

Two file (golang-migrate compatible)

```
20260701093000_add_orders.up.sql    → CREATE TABLE ...
20260701093000_add_orders.down.sql → DROP TABLE ...
```

Naming rules that actually bite:

Rule	Consequence if broken
Version prefix is all digits	File is ignored
Separator is <code>_</code> or <code>-</code>	File is ignored
Version must be unique	Hard error naming both files
Single file needs <code>-- migrate:up</code>	Error: no marker found

Set the default with `MIGRATION_STYLE` in `.env` (`one` / `dbmate` , or `two` / `golang-migrate`).
The `--style` **flag** itself takes only `one` or `two` .

13. Check status before you act

`status` is the "what is about to happen" command.

```
scm status
```

Version	Label	Status	Down?
20260601120000	create_users_table	applied	yes
20260602083000	add_email_index	applied	yes
20260603094500	add_orders_table	pending	yes
20260604110000	add_audit_log	pending	no
20260530000000	<file missing>	orphaned	?

```
4 migrations: 2 applied, 2 pending.
1 orphaned (applied in DB but no file on disk).
```

Colour coded in a terminal, plain when piped: **dim** = applied, **green** = pending, **yellow** = orphaned.

The `Down?` column is a pre flight check — a `no` there means that migration cannot be rolled back. Notice it *before* you apply, not after.

14. Apply with up

```
scm up          # apply everything pending
scm up 1        # apply only the next one
scm up --dry-run # print the SQL, change nothing
```

```
Applying 20260701093000_add_orders_table ... OK
```

- Each migration runs **in its own transaction**; a failure rolls that migration back and stops immediately. Migrations already applied stay applied.
- Statements are split by a comment and string aware parser, so semicolons inside `-- comments`, `/* blocks */`, and `'literals'` are safe.
- PostgreSQL files are sent whole, so `$$ ... $$` blocks work. SQL Server also splits on `GO`.

Habits to teach: `--dry-run` on anything you have not run before, and `up 1` when applying to production so a bad migration fails fast and alone.

15. Roll back with down

```
scm down          # roll back the most recent migration
scm down 3        # roll back the last three
scm down --dry-run # preview the rollback SQL
```

If a migration has no down script, `down` **errors out rather than silently skipping** — so a missing `down` is a decision you make when you write the file, never a surprise at rollback time.

Practise this. A `down` that has never been run is not known to work. Rolling forward and backward once in dev is the cheapest possible test.

16. Jump to a version with goto

```
scm goto 20260601120000
scm goto 20260603110000 --dry-run
```

`goto` puts the database at **exactly** the named version, whichever direction that requires:

- rolls back, newest first, anything applied above the target
- applies, oldest first, anything pending at or below the target
- exits with no changes if you are already there

If any migration on the rollback path has no down script, `goto` refuses **before** making any change. Ideal for reproducing a bug at an older schema, or for pinning an environment to a known point.

17. Inspect with show, history, report, ddl

Four read only commands. None of them change anything.

```
scm show <version> # print the up and down SQL (no DB connection needed)
scm history        # every applied migration, who applied it, and when
scm report         # activity summary, per user breakdown, recent activity
scm ddl           # verify the tracking table has all expected columns
```

Version	Label	By	Applied At

```
20260601120000 create_users_table alice 2026-06-01 12:03:44+00
20260602083000 add_email_index bob 2026-06-02 09:11:07+00
```

Migration Activity Report

```
database      public.schema_migrations
applied       12
pending       2
first applied 2026-06-01 12:03:44+00 20260601120000 create_users_table
last applied  2026-07-09 08:15:02+00 20260709081502 add_orders
```

By user

User	Count	First	Last
alice	8	2026-06-01 12:03:44+00	2026-07-09 08:15:02+00
bob	4	2026-06-12 09:20:11+00	2026-06-30 14:02:55+00

`scm show` before `scm up` is the review step: read the SQL you are about to run, without touching the database.

18. Adopt an existing database with baseline

The classic adoption problem: the schema already exists, built by hand over years. You do not want to run those migrations, you just need SCM to know they are done.

```
scm baseline # mark every migration as applied
scm baseline 20260602083000 # mark up to a specific version
```

```
Baselined 20260601120000_create_users_table
Baselined 20260602083000_add_email_index
```

2 migration(s) marked as applied.

Rows are written to the tracking table and **no SQL is executed**. Afterwards, `scm up` applies only what was created after the baseline point.

On PostgreSQL, if an older `schema_migrations` lives in a non `public` schema, scm will now look in `public` and see nothing. Either set `MIGRATIONS_SCHEMA` to the old schema, move the table, or re establish state with `baseline`.

19. The careful commands: undo, repair, reset

Grouped deliberately — these delete things.

Command	What it removes	When to use
<code>scm undo</code>	Pending migration files, interactively selected	You created a file you no longer want
<code>scm repair</code>	Orphaned tracking rows with no file on disk	A file was deleted after being applied
<code>scm reset --force</code>	All tracking rows and every <code>.sql</code> file	Wiping a dev environment only

Pending migrations:

1. 20260603094500_add_orders_table
2. 20260604110000_add_audit_log

Enter number(s) to undo (e.g. 1 or 1,3 or 1-3), blank to cancel: 2

Deleted 20260604110000_add_audit_log.sql

reset is **destructive and irreversible**. It does **not** touch your schema — only history and files — which is exactly why it is dangerous: the database keeps the tables while SCM forgets they were ever created. Dev only, and check `scm who` first.

`undo` is for changes not yet applied. To reverse an **applied** migration, use `scm down`.

20. Many databases, one migration set

`--db <name>` scopes every command to one database:

```
scm --db hana-t2 status
scm --db hana-t2 new add_orders_table
scm --db hana-t2 up
```

It resolves automatically:

- `--env-file` → `.env.hana-t2`

- `--path` → `./migrations/hana-t2` **if that folder exists**, otherwise the shared `./migrations`

That fallback is the useful part: **one** set of migration files can be promoted across dev, stage, and prod, each selected by its own `.env.<name>`. Create a per database folder only when a database genuinely needs its own migration set.

<code>.env.hana-t1</code>	<code>migrations/</code>	← shared set, promoted everywhere
<code>.env.hana-t2</code>	<code>hana-t2/</code>	← only if this DB needs its own
<code>.env.ods</code>	<code>ods/</code>	

With `--db`, scm loads `.env.<name>` first and the base `.env` second, so the per database file wins on `DATABASE_URL` while any shared settings still come from the base `.env`.

Concurrency warning: scm takes no advisory lock. Two `up` runs against the same database at the same time will collide, and on MySQL any DDL already executed is permanent. Serialize migrations in CI.

21. Writing safe migrations

The rules that keep migrations boring:

- **Always write a `down`**, or comment why the change is irreversible.
- **Guard with `IF EXISTS` / `IF NOT EXISTS`** so a re run after a partial failure does not error.
- **Never edit an applied migration.** Once it has run anywhere, it is immutable. Write a new migration to correct it.
- **No secrets, passwords, or PII** in migration SQL. It is committed to git forever.
- **One logical change per file.** Small migrations fail small.

Adding a column to a populated table:

```
-- Fails on a table with rows: NOT NULL and no default
ALTER TABLE users ADD COLUMN tier varchar(20) NOT NULL;

-- Safe: nullable now, backfill, constrain in a later migration
ALTER TABLE users ADD COLUMN tier varchar(20);

-- Also safe: a default applies retroactively
ALTER TABLE users ADD COLUMN tier varchar(20) NOT NULL DEFAULT 'free';
```

Destructive operation	Risk	Mitigation
<code>DROP TABLE</code>	Permanent data loss	<code>IF EXISTS ;</code> <code>down</code> must recreate it
<code>DROP COLUMN</code>	Permanent data loss	Deploy the app change first
<code>TRUNCATE</code>	Empties the table	Rarely belongs in a migration
<code>DELETE</code> without <code>WHERE</code>	Empties the table	Always add a <code>WHERE</code>
Type change	May truncate data	Test on a production sized copy
Rename table or column	Breaks the running app	Coordinate with the code deploy

Keep **data** migrations separate from **schema** migrations, and batch large updates to avoid long locks.

22. Database specific gotchas

Engine	DDL transactional?	Watch out for
PostgreSQL	Yes	<code>CREATE INDEX CONCURRENTLY</code> cannot run in a transaction — give it its own file
MySQL / MariaDB	No	A failed migration can partially apply and cannot be auto rolled back — one DDL change per file
SQL Server	Yes	Use <code>GO</code> to separate batches that must run independently
SQLite	Yes	No <code>ADD COLUMN IF NOT EXISTS ;</code> no <code>DROP COLUMN</code> before 3.35
Oracle / HANA / Snowflake / cTree	Varies	Fewer <code>IF EXISTS</code> guards available; check the dialect

The MySQL point deserves emphasis — it is the most common source of "the migration failed but half of it is still there." On MySQL, keep migrations tiny.

23. Credentials, permissions, and CI

Credentials

- `DATABASE_URL` lives in `.env`, and `.env` lives in `.gitignore`.
- Use a **dedicated migration user** — not the application runtime user, not a superuser.
- Avoid `--url`; it exposes the password in shell history and process lists.

```
-- PostgreSQL migration user
GRANT CONNECT ON DATABASE mydb TO migrator;
GRANT USAGE, CREATE ON SCHEMA public TO migrator;
GRANT INSERT, DELETE ON schema_migrations TO migrator;

-- MySQL migration user
GRANT CREATE, ALTER, DROP, INDEX, REFERENCES ON mydb.* TO 'migrator'@'%';
GRANT INSERT, DELETE ON mydb.schema_migrations TO 'migrator'@'%';
```

In CI/CD

- Mask `DATABASE_URL` so it never reaches the logs.
- Run `scm status` before `scm up` so the log records exactly what was applied.
- Consider `scm up 1` per step so a bad migration fails fast.
- Serialize the migration job. Only one `up` at a time per database.

24. AI ready: using SCM with Claude

SCM is AI ready — you do not have to teach the AI anything. Four things make that true:

- **The rules ship with the repo.** `scm.md` contains a *Rules for Claude* section, so any assistant working in this codebase reads them as context and follows them without being prompted.
- **The tool documents itself.** `scm learn` prints the full command reference, so an AI can look up correct usage instead of guessing at flags.
- **Nothing proprietary to learn.** Migrations are plain SQL in the dbmate format that models already know, and the command set (`new`, `status`, `up`, `down`) is small and conventional.
- **Safe by construction.** `--dry-run`, `scm show`, and a required `down` mean an AI's proposal can be reviewed before anything touches the database.

The rule that makes AI assisted schema work safe:

Claude never runs DDL directly. No `CREATE` , `ALTER` , or `DROP` through a database tool. It must produce an SCM migration file instead.

What Claude does:

1. Creates a git branch (`migration/<name>`).
2. Runs `scm new <name>` — never hand names a file.
3. Writes the `up` **and** `down` SQL, with `IF EXISTS` guards.
4. Reads the live schema through MCP (tables, fields, indexes) to get the SQL right.
5. Hands you the file path and reminds you to run `scm up` .

What stays with the human:

- **Reviewing the generated SQL.** `scm show <version>` exists for this.
- **Running `scm up`** . Claude writes; you approve and apply.

Why it matters: every AI made change becomes a reviewed SQL file in git with a rollback, instead of an invisible one off command nobody can find later. Same files, same commands, whether a person or Claude wrote them — SCM is the shared contract.

25. Hands on lab

Ten minutes, no server required. SQLite is built into Python.

```
# 1. A scratch project
mkdir scmlab && cd scmlab
scm init

# 2. Point it at a local SQLite file (two slashes = relative path)
#   .env → DATABASE_URL=sqlite://demo.db

# 3. Create a migration
scm new create_users_table
```

Fill in the generated file:

```
-- migrate:up
CREATE TABLE IF NOT EXISTS users (
  id    INTEGER PRIMARY KEY,
  email TEXT NOT NULL
);
```

```
-- migrate:down
DROP TABLE IF EXISTS users;
```

Now walk the whole loop and watch what each command reports:

```
scm who          # confirm the target
scm status       # 1 migrations: 0 applied, 1 pending
scm up --dry-run  # see the SQL, change nothing
scm up          # Applying 20260729161235_create_users_table ... OK
scm status       # 1 migrations: 1 applied, 0 pending
scm history      # your username and a timestamp
scm report       # activity summary
scm ddl          # all expected columns present
scm down         # roll it back
scm status       # pending again
scm up          # and forward once more
```

Stretch goals: add a second migration, use `goto` to hop between the two, delete a pending file with `undo` , then delete an applied file from disk and fix the orphan with `repair` .

26. The five rules to remember

1. **If it is not a migration, it did not happen.** Schema changes only through SCM — for people and for AI.
2. **Check `scm who` before you act.** The right SQL on the wrong database is still an outage.
3. **Always write the `down` ,** and roll it back once in dev to prove it works.
4. **Never edit an applied migration.** Write a new one.
5. **`--dry-run` and `status` are free.** Use them before every `up` .

Where to go next

Need	Go to
Full command reference	scm.md
Installing scm on its own	scm-readme.md
Teaching outline for trainers	scm-help.md
Cheat sheet in your terminal	<code>scm learn</code> (or <code>scm help</code>)

Questions?